

ModIntel Test Plan Document (TPD)

Contents

- [List of Tables](#)
- [List of Figures](#)
- [1. Introduction](#)
 - [1.1 Purpose of the document](#)
- [2. Features to be tested/not to be tested](#)
 - [2.1 Features to be tested](#)
 - [2.2 Features not to be tested](#)
- [3. Pass/Fail criteria](#)
- [4. Approach/Strategy](#)
- [5. Test cases with specifications](#)
- [Reference](#)

List of Tables

- Table 1: Test case specification for User Authentication (Unit)
- Table 2: Test case specification for AI Inference Prediction (Unit)
- Table 3: Test case specification for Inter-Service Communication (Integration)
- Table 4: Test case specification for API Data Retrieval (Integration)
- Table 5: Test case specification for End-to-End Gateway WAF Routing (System)

List of Figures

- No figures in this document.

1. Introduction

The ModIntel platform is an intelligent web application firewall and monitoring system consisting of multiple microservices including Authentication, Review API, Health Aggregator, Log Collector, and an ML Inference Engine. This document outlines the comprehensive testing strategy applied across the entire stack.

1.1 Purpose of the document

The purpose of this Test Plan Document (TPD) is to describe the testing scope, approach, resources, and schedule of testing activities for the ModIntel system. It provides a formal framework for verifying that the software meets its design specifications and functional requirements through Unit, Integration, and System testing.

2. Features to be tested/not to be tested

2.1. Features to be tested

- User Authentication:** JWT token generation, password hashing, rate limiting, and RBAC (Role-Based Access Control) enforcement.
- Microservice APIs:** Review API endpoints, alert pagination, and configuration retrieval.
- Machine Learning Inference:** Payload feature extraction, ONNX model prediction, anomaly scoring, and model calibration metrics.
- Service Integration:** Cross-container network communication, database read/writes, and aggregate health monitoring.
- WAF Gateway Flow:** Proxy routing via Caddy, Coraza CRS rule execution, and blocking of known attack vectors (SQLi, XSS, Path Traversal).

2.2. Features not to be tested

- Third-party Service Uptime:** External database managed availability (e.g., MongoDB Atlas if used).
- Frontend Dashboard UI Rendering:** Cross-browser CSS/HTML visual regression testing (beyond basic route reachability).
- Physical Network Infrastructure:** Underlying host OS and hardware load balancing.

3. Pass/Fail criteria

- Pass Criteria:** A test is considered "Pass" if the actual output matches the expected output exactly, the system does not crash under test loads, and correct HTTP status codes are returned (e.g., 200 OK for success, 401 Unauthorized for bad credentials, 403 Forbidden for blocked attacks).
- Fail Criteria:** A test is considered "Fail" if the system throws an unhandled exception (HTTP 500), allows unauthorized access, fails to block a known malicious payload, or returns data that does not match the strict test specifications.

4. Approach/Strategy

The testing strategy utilizes a standard testing pyramid:

- Unit Testing:** Isolated tests on individual Go and Python functions (e.g., JWT parsing, feature extraction) using mock data. Executed via `go test` and `pytest`.
- Integration Testing:** Tests ensuring that internal microservices interact correctly (e.g., Auth service reading from MongoDB, Log collector sending data to the Inference Engine). Executed via `go test -tags integration` within a Docker Compose environment.
- System Testing:** End-to-End (E2E) tests simulating real user and attacker traffic entering through the public WAF gateway, verifying the entire pipeline from attack detection to dashboard alert generation. Executed via `go test -tags system`.

5. Test cases with specifications

Table 1: Test case specification for User Authentication (Unit)

Name: User Login Validation

Purpose: Verify that the authentication service validates credentials correctly and blocks unauthorized access.

Test Data:

- Email (Valid Email, Invalid Email, Empty)
- Password (Valid Password, Invalid Password, Empty)

Input	Expected result	Data	Actual output	Pass/fail
All valid fields	"Login successful, token generated"	Email: admin@modintel.local Password: ChangeMe123!	"Login successful, token generated"	Pass
Invalid Email and all other fields are valid	"Invalid credentials"	If Email is not registered Or If Email format is invalid	"Invalid credentials"	Pass
Invalid Password and all other fields are valid	"Invalid credentials"	If Password does not match hash	"Invalid credentials"	Pass
Empty Email or Password	"Bad Request: Missing fields"	Any empty field for Email or Password	"Bad Request: Missing fields"	Pass

Table 2: Test case specification for AI Inference Prediction (Unit)

Name: Attack Probability Scoring
Purpose: Ensure the ML model correctly extracts features and assigns an attack probability to HTTP requests.
Test Data:

- Request Features (SQLi Payload, Normal Traffic Payload)

Input	Expected result	Data	Actual output	Pass/fail
Normal Traffic Features	Attack Probability < 0.2 Priority: None	URI: /index.html Body: empty	Attack Probability: 0.05	Pass
SQLi Payload Features	Attack Probability > 0.8 Priority: P1 or P2	URI: /login?id=1' OR '1'='1	Attack Probability: 0.92 Priority: P1	Pass
Missing Features	"Validation Error"	Empty URI	"Validation Error"	Pass

Table 3: Test case specification for Inter-Service Communication (Integration)

Name: Aggregate Health Monitoring
Purpose: Verify that the Health Aggregator can successfully ping all dependent microservices.
Test Data:

- Network status of internal containers

Input	Expected result	Data	Actual output	Pass/fail
All containers running	"All services return status OK"	HTTP GET to /api/health/aggregate	"review-api: ok, proxy-waf: ok, auth: ok"	Pass
Database offline	"Database shows as degraded"	Kill MongoDB container	"review-api: degraded"	Pass

Table 4: Test case specification for API Data Retrieval (Integration)

Name: Review API Alert Pagination
Purpose: Verify the Review API retrieves alerts from the database using cursor-based pagination.
Test Data:

- Limit (Valid int, Too High)
- Cursor (Valid ObjectID, Empty)

Input	Expected result	Data	Actual output	Pass/fail
Valid Limit, Empty Cursor	"Returns first N records + next cursor"	Limit: 50 Cursor: ""	"50 records returned"	Pass
Valid Limit, Valid Cursor	"Returns next N records after cursor"	Limit: 25 Cursor: 64a2b...	"25 records returned"	Pass
Invalid Limit (Too High)	"Bad Request: Limit exceeded"	Limit: 1000	"Bad Request"	Pass

Table 5: Test case specification for End-to-End Gateway WAF Routing (System)

Name: WAF Attack Simulation and End-to-End Routing
Purpose: Verify that the WAF Proxy blocks malicious traffic while routing normal user dashboard requests correctly.
Test Data:

- HTTP Requests (Normal traffic, Malicious payloads)

Input	Expected result	Data	Actual output	Pass/fail
Normal Traffic via WAF	"200 OK or 3xx Redirect"	GET /events or /	"200 OK"	Pass
SQLi Payload via WAF	"403 Forbidden (Blocked by Coraza)"	GET /search?q=1%27%20OR%201%3D1--	"403 Forbidden"	Pass
XSS Payload via WAF	"403 Forbidden (Blocked by Coraza)"	GET /page?q=<script>alert(1)</script>	"403 Forbidden"	Pass
Full Auth Flow via Gateway	"Login -> Access Protected -> Logout Succeeds"	Valid credentials -> /me -> logout	"Flow completed successfully"	Pass

Reference

[1] IEEE Standard for Software Test Documentation, IEEE Std 829-2008.
[2] ModIntel Architecture and API Documentation (Internal).